

WEEK 1



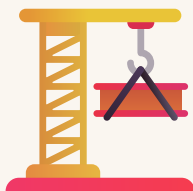
Functions with Inputs

One function, many towers

함수에 숫자를 넣어 매번 다르게 짓기

BIG PICTURE

8 weeks → your own game 🎮



Teach the agent one skill, then reuse it everywhere.

작은 함수들을 모아 게임을 만듭니다

HOW TO COUNT

Count the axis — one block, one number



1 Stand on the start block — that spot is $(0, 0, 0)$.

2 $x \rightarrow$ walk **right**. Add **+1** for every block.

3 $y \rightarrow$ jump **up**. Add **+1** for every block.

4 $z \rightarrow$ walk **forward**. Add **+1** for every block.

Say them in order every time: $(x, y, z) = \text{across, up, forward}$.

한 칸 움직이면 +1 — 오른쪽 x , 위로 y , 앞으로 z . 순서는 늘 (x, y, z)

TYPE IT IN

Put the numbers into Minecraft



^x
across →

^y
up ↑

^z
forward ↗

```
place STONE at (x, y, z)
```

- ✓ Open **Code Builder** (press **C** in the world)
- ✓ Type the numbers in order — **(x, y, z)**
- ✓ Press **Run ▶** — your block appears at that spot

코드 빌더(C) 열고 → (x, y, z) 순서로 숫자 넣고 → Run ▶

CONCEPT

What is a function?

A named block of steps you can **reuse** as many times as you want.

```
def build_tower():  
    agent.place(STONE, FORWARD)  
    agent.move(UP, 1)
```

Define once. `build_tower()` runs it again and again.

한 번 정의 → 여러 번 호출

CONCEPT

A parameter — an input

A parameter is a number the function uses **inside**.

```
def build_tower(height):  
    for i in range(height):  
        agent.place(STONE, FORWARD)  
        agent.move(UP, 1)
```

height changes each call — same code, different tower.

매개변수 = 함수가 안에서 쓰는 입력값

CONCEPT

Calling with different numbers

`build_tower(2)`
tower 2 tall

`build_tower(5)`
tower 5 tall

`build_tower(3)`
tower 3 tall

`build_tower(8)`
tower 8 tall

하나의 함수, 매번 다른 높이

WATCH IT!

Call → run → result

```
build_tower(3)
```

→

↓ call

```
def build_tower(height):  
    for i in range(height):  
        agent.place(STONE, FORWARD)
```

↑ done

호출하면 함수 안이 실행돼요

CONCEPT

return — give an answer back

A function can **give back** a value instead of building.

```
def count_steps(distance):  
    return distance + 2  
  
steps = count_steps(4)  
player.say(steps)    # 6
```

return = 값을 돌려주기

PREDICT 🤔

How tall is the tower?

Predict first — then click reveal

```
def build_tower(height):  
    for i in range(height):  
        agent.place(STONE, FORWARD)  
        agent.move(UP, 1)  
  
build_tower(3)
```

Reveal output 🔍 reveal

VOCAB

New words today

function a named block of code you can reuse

parameter an input a function uses inside

call running a function by name + ()

return the value a function gives back

함수

매개변수

호출

반환



What's wrong with this code?

Goal: build a tower 4 tall — but **nothing** happens.

buggy

```
def build_tower(height):  
    for i in range(height):  
        agent.place(STONE, FORWARD)  
        agent.move(UP, 1)
```

Think first — then click next.



DEBUG

The bug

Bug: defining a function only **teaches** it. You never **called** it, so nothing runs. Add a call.

fixed

```
def build_tower(height):  
    for i in range(height):  
        agent.place(STONE, FORWARD)  
        agent.move(UP, 1)
```

```
build_tower(4)
```

YOUR TURN!

Build a tower village

✓ Write one function with a `height` parameter

✓ Use `height` inside the `range` loop

✓ Call it 3+ times with different numbers

✓ Watch towers of different sizes appear

✓ Send a photo of your skyline



QUIZ

QUESTION 1 OF 3

What is a **parameter**?

The name of the agent.

An input the function uses inside, like `height` .

A block in Minecraft.

The world file.



QUIZ

QUESTION 2 OF 3

You wrote `def build_tower(height):` but nothing builds. Why?

height must be capitalised.

You defined it but never **called** it.

You need two parameters.

Functions can't build towers.



QUIZ

QUESTION 3 OF 3

What does `return distance + 2` do?

Places 2 blocks.

Turns the agent.

Nothing — return is invalid.

Gives a value back to whoever called the function.

QUIZ

Your score

0 / 3



Re-read the slides. You'll get it.

↺ retake



One function, many builds!

Next: run your code from chat.

다음 주: 채팅 명령으로 실행하기